

Neighbour-Link - Team38

Neighbour-Link - Team38 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Social Media · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): An application to revive local neighbourhood cohesion by matching residents through shared hobbies, proximity and community events. Rather than relying purely on remote servers, it uses an **on-device C++ location and matching engine** to process proximity and pair user profiles locally. To protect privacy and comply with PDPA, exact 6-digit postal codes are **masked into 2-digit sector zones on-device** before sync, allowing safe offline meetups without exposing home addresses. Four technical features: **Anonymized Sector Geofencing** (PDPA-compliant location processing), **Smart Mutual Interest & Proximity Matcher**, **Live Location & Event Tracker**, **Instant Live Event Alerts & Sync**. Custom tool: a **Dynamic Hobby & Event Schema Editor** (author hobby categories, matching criteria and event parameters with live in-app preview). Four design features: hobbies discovery feed, neighbourhood event creation tool, automated interest-matching feed, and a gamified reputation & local reward dashboard.

Adjustments required for approval: *(the earlier blocker — that matching be the team's own load-bearing C++ service rather than a Firebase query — is now resolved by the on-device C++ engine; two minimal conditions remain)*

- **Safety-by-design for connecting strangers offline.** The sector-masking handles *location* privacy well; add the *interpersonal* safety surface too — verified accounts, report + block, per-user visibility controls, and a **moderation / report-review dashboard** (fold this into the custom tool), plus in-app public-place guidance for meetups. This is a graded surface, not an afterthought.
- **Rewards stay in-app points — no real money.** Compute reputation/points **server-side** (anti-gaming) and keep the "local reward" as an in-app points ledger with redeemable codes; do **not** process real payments or run an e-

commerce/discount-settlement flow, which would push the project into Fintech and out of scope for this trimester.

References:

- [Nextdoor](#) — the neighbourhood network (verified neighbours, local events, "turn shared interests into real-life connections")
- [Meetup](#) — find local groups & events near you around shared interests
- [Bumble For Friends](#) — match nearby people by shared interests, with photo verification and IRL meetups

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

Core Tech Requirement	Neighbour-Link
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	♦
3D (strong students only)	✗
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — the on-device C++ location/matching engine, sector geofencing and push alerts are mobile-first (device sensors/GPS + on-device processing favour a native build), with a mobile-web front-end for reach; the Native PC build is the development/tools vehicle (device: Pixel 9a). **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: Hobbies-discovery feed, interest-match feed, event-creation and schema-editor screens, a live location/event proximity view (masked sector zones) and a reputation/reward dashboard (OpenGL ES for the custom feed cards, geofence/proximity view and reputation charts). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: The on-device C++ engine (proximity/distance updates, on-device sector-masking, the user-matching state machine) plus a server tier for sync, notification dispatch, the reputation/points ledger and the moderation/report queue. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — matching runs on-device; the Linux server seam is set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing piece is the **on-device C++ engine** — a real-time update loop that ticks proximity/distance calculations, applies the PDPA sector-masking transform on-device (6-digit → 2-digit sector) before any sync, and drives the user-matching state machine (interest + masked-proximity scoring). This is genuine C++ systems work, not a database query.
- *Data-driven:* hobbies, interest tags, matching criteria/weights, event parameters and reputation/reward tiers are all hot-reloadable JSON authored by the Schema Editor — no hard-coded hobbies, event types or scoring constants.
- *2D:* custom-rendered feed cards, a masked-sector proximity/geofence view, and reputation/engagement charts (custom-rendered, no chart libs) — the domain-adapted 2D surface for a Social Media app. *3D:* not needed.
Networking: Live Location & Event Tracker + Instant Live Event Alerts & Sync — push (FCM) + an in-app event bus + server sync of events and masked presence.
- *Custom tool:* the **Dynamic Hobby & Event Schema Editor** (author hobby/tag/event schemas and matching criteria with live in-app preview), extended with a **moderation / report-review dashboard** (PC-side authoring/ops).
- *External DB:* Firebase stores users, hobbies, events, the reputation ledger and reports (masked location only). *Auth »:* verified accounts are safety-

critical here. *Analytics* »: optional engagement funnels — which hobbies/
events convert to real-life meetups.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window and input with a fixed-timestep loop driving the on-device proximity tick, the sector-masking pass and the match state machine; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G9 matching engine and M1G12 alerts run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with users, hobbies, interest tags, events, matching weights and reputation/reward rules defined in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M1G7 (hobby/event schemas), M1G9 (scoring reads this model), M2G2 (the Schema Editor edits this data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the match index, geofence state and alert queue, Tracy profiling, Catch2 stress tests re-ranking a synthetic neighbourhood (tens of thousands of users/events across many sectors at once), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the on-device frame/latency budget M2G1 validates.

- **G7 Data-Driven Content Models** — hobby, event and reputation schemas as JSON: the data the Dynamic Schema Editor authors.

Tech & dependencies

- **Tech:** Versioned JSON schemas for hobbies/interest tags, event parameters and reward tiers, with schema validation and hot-reload.
- **Prerequisites:** G2 (data-driven architecture).
- **Feeds into:** M1G9 (scoring features), M2G2 (Schema Editor), M2G8 (recommendation features).

- **G9 Matching / Ranking Engine — Foundation** — the on-device C++ matcher: score neighbours/events by shared interest + masked-sector proximity.

Tech & dependencies

- **Tech:** Priority queues and weighted scoring (interest-tag overlap + masked 2-digit-sector distance), the on-device sector-masking transform, a maintained nearby index and incremental re-ranking (custom C++) — runs locally, not as a Firebase query.
- **Prerequisites:** G2 (user/hobby/event model), G1 (tick).
- **Feeds into:** M2G8 (recommendations II), the discovery feed and match alerts; M2G16 (privacy model).

- **G12 Notification System — Foundation** — Instant Live Event Alerts & Sync: new nearby match, event invite, reputation change.

Tech & dependencies

- **Tech:** Firebase Cloud Messaging (push) plus an in-app event bus and a web service worker; notification schema, deep-link targets and delivery tracking.
- **Prerequisites:** G1 (loop/event bus); Firebase integration seam.
- **Feeds into:** M2G7 (push/identity completion) and the Live Event Tracker.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the on-device match/geofence passes, ASan on MSVC, memory pools for the nearby index, and Emscripten WASM validation for the mobile-web target plus on-device Pixel 9a checks; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native

Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The web/mobile deliverable and headroom for larger neighbourhoods on-device.

- **G2 ★ Content Editor — Core** — the Dynamic Hobby & Event Schema Editor + moderation/report dashboard: author hobby/tag/event schemas and matching criteria with live preview, tune reward tiers, action reports.

Tech & dependencies

- **Tech:** ImGui editor authoring hobby/tag/event schemas and matching-weight/reward-tier tables as JSON, with live in-app preview, round-trip serialization and an undo/redo command stack; a report-queue review pane.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL renderer (live feed/geofence preview).
- **Prerequisites:** G2 (data model), M1G7 (content schemas).
- **Feeds into:** M2G8 (weights it tunes), M2G16 (safety actions applied through it).

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the native build plus the Emscripten web build artifact via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every Neighbour-Link goal.

- **G8 Ranking / Matching Engine II** — recommendations: rank neighbour/event suggestions by interest + masked proximity + reputation.

Tech & dependencies

- **Tech:** Scoring models (weighted features, cosine similarity on interest-tag vectors), candidate generation from the on-device nearby index, and evaluation metrics; reputation as a scoring feature.
- **Prerequisites:** M1G9 (matching foundation), M1G7 (feature schemas).

- **Feeds into:** The discovery feed, "neighbours you may know" and event/meetup suggestions.
- **G7 Firebase — Auth, Push & Analytics** — verified accounts, push delivery, masked-data sync and the reputation ledger.

Tech & dependencies

- **Tech:** Firebase Auth (verified accounts — safety-critical), Firestore/ Realtime DB for hobbies/events/reputation (masked location only), Cloud Messaging for push and Analytics — REST API from C++, JS SDK on the web front-end.
- **Prerequisites:** G2 (data model to sync), M1G12 (notification foundation).
- **Feeds into:** M2G16 (identity-backed safety controls) and the points/ reputation ledger.
- **G16 Privacy & Safety Controls** — the PDPA/geofencing privacy model + blocking, visibility and report actions.

Tech & dependencies

- **Tech:** On-device sector-masking as the privacy boundary (only masked location leaves the device), block lists, per-user visibility flags, a report-to-action flow and Firebase security rules; ties to the moderation dashboard.
- **Prerequisites:** M2G7 (auth/identity), M2G2 (moderation dashboard), M1G9 (masking transform).
- **Feeds into:** Safe discovery, safe offline meetups and trustworthy reputation.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art (8) — the four design features (feed, event creation, interest matching, reputation & reward) are a strong single-designer load, and the art surface (feed cards, hobby/interest iconography, reputation badges & reward tiers, event/geofence imagery) is a utility social-app surface one artist can cover (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). Add a 2nd artist only if a richer illustrated hobby/badge set becomes a selling point.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the discover → match → create/join event → earn reputation loop, plus the Schema Editor and the in-app points/reward economy — anti-gaming)
- M1D2 Prototype (clickable flow proving the interest-match → event-join path feels worthwhile)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (matching weights and reward tiers tuned via the team's JSON, no code changes)

- **UI/UX M1** (graded by the UI/UX instructor):

- M1U1 ★ UI Standards & Wireframe Baseline — screen-flow map and readability standards for the feed, matching, event-creation and reputation screens.
- M1U2 UX Flows & Wireframes — the discover → match → join-event and hobby-creation journeys.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the required Social Media screens (feed, profile, event, notifications, report/block) navigable unaided.
- M2U2 ★ UX Validation & Final Polished UI — touch-first layouts, alert affordances and the masked-sector proximity view validated on the Pixel 9a / mobile-web target.

- **Art M1:** 1 artist (utility social-app surface)
 - M1A1 ★ (mockups: hobbies-discovery feed + event-creation screen)
 - M1A4 UI Art Concepts & Typography (feed cards, hobby/interest icon set, reputation/reward visual language)
 - M1A6 Early Asset Production (first production-ready icon & badge set)
- **Art M2:**
 - M2A1 ★ (polished feed, event and reputation screens)
 - M2A4 UI Art — States & Feedback (match found, event full, reputation up/down, reward unlocked)
 - M2A5 Illustration & Iconography — Full Set (complete hobby/interest icons, reward-tier badges, empty/onboarding states)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial UI/alert SFX and light ambient BGM from the week-5 needs list, present & working in the prototype (match found, event invite, reputation change), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback (overlapping alert/UI SFX).
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Social Media floor (subtle UI/alert SFX + optional light ambient BGM in the showcase build; audio is not load-bearing here).
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing for stacked alerts).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase, prototype demo, team post-mortem (wk14)